

Robust Radial Normalization for Solar Filament Instance Segmentation

A Solution to the Solar Filament Segmentation Challenge 2026

ILoveBuns

China

1280401315+ILoveBuns@users.noreply.github.com

ABSTRACT

We present a deterministic, CPU-friendly instance-segmentation pipeline for solar filaments. The method removes center-to-limb brightness variation with robust radial normalization, identifies dark filament candidates relative to the visible solar disk, separates them into connected components, filters small components, and emits compressed COCO run-length encodings. Our strongest official Kaggle submission is platform reference 55113085, with a public score of **0.48**. This is higher than our later Mask R-CNN (0.31) and frozen public YOLO plus U-Net (0.34) experiments. The classical pipeline is therefore the selected final method. Its principal advantages are reproducibility, low compute cost, and transparent failure modes.

1 INTRODUCTION

This report describes our submitted solution to the Solar Filament Segmentation Challenge 2026 [1], organized by the Earth-Space AI Research Lab. The challenge uses GONG H- α observations and evaluates pixel-level masks for individual solar filaments with Panoptic Quality (PQ) [2]. In addition to overlap, the evaluation penalizes false detections, missed instances, fragmentation, and over-merging.

Solar full-disk images exhibit strong center-to-limb brightness variation. A fixed global threshold therefore confounds the dark limb and background with actual filament material. Our contribution is an intentionally simple, inspectable correction and grouping pipeline. We do not claim a new neural architecture. Instead, we report the method that produced our highest verified platform score, distinguish official scores from local checks, and provide an end-to-end notebook that needs no weights or private supporting files.

2 METHODOLOGY

2.1 Design objective

The final system was chosen to optimize the complete competition objective, not only binary foreground overlap. A useful prediction must localize dark material, partition it into plausible instances, avoid duplicate pixels across instances, and serialize every mask in the required format. These constraints favor a small number of explicit stages whose effects can be inspected independently. The resulting prediction path is summarized below.

grayscale image $\rightarrow$ disk support $\rightarrow$ radial profile $\rightarrow$
normalized contrast $\rightarrow$ adaptive threshold $\rightarrow$ connected
components $\rightarrow$ area/rank filtering $\rightarrow$ compressed COCO
RLE

Figure 1: Deterministic inference pipeline. Each image is processed independently and produces zero or more disjoint instance masks.

2.2 Input and radial normalization

Official JPEG inputs are converted to grayscale. For an image of height H and width W , we use its geometric center and a disk radius $r_d = 0.48 \min(H, W)$. Disk pixels are divided into 96 radial bins. For every bin b , the background profile P_b is its upper-quartile intensity. The upper quartile is less sensitive than the mean to dark filament contamination. A pixel x in bin $b(x)$ is normalized as

$$N(x) = \frac{I(x)}{\max(P_{b(x)}, 10^{-6})}. \quad (1)$$

This operation removes broad radial illumination variation while preserving localized dark structures. Pixels outside the estimated disk are never considered filament candidates.

2.3 Candidate extraction and instance generation

Within the disk, we compute the 25th percentile $q_{0.25}$ of normalized intensity and set

$$\tau = \min(q_{0.25}, 0.95). \quad (2)$$

Pixels satisfying $N(x) \leq \tau$ form the candidate foreground. Four-connected component labeling converts this foreground into instances. During submitted inference, components smaller than 20 pixels are removed and no more than the 64 largest components per image are retained. Accepted masks are encoded as compressed COCO RLE [3]. Sorting input paths and component sizes makes the output deterministic for fixed data and dependencies.

The complete prediction path is implemented in `solarfil/infer.py` and `solarfil/segment.py`. The supplied final notebook installs the pinned environment, finds the official test directory, runs every test image, writes the required two-column CSV, checks unique identifiers and nonempty encodings, and decodes a sample RLE to verify its dimensions. It performs no training, downloads no model, and calls no inference service.

2.4 Algorithm and invariants

For each sorted input path, Algorithm 2 maintains a single grayscale image, normalized array, Boolean candidate map, and integer label map. The label map is reused to encode one selected component at a time, avoiding a dense stack of up to 64 full-resolution masks. Component identifiers are sorted by decreasing area; Python’s stable ordering provides deterministic tie handling for a fixed SciPy labeling result.

- (1) Read one JPEG as grayscale and estimate the circular disk.
- (2) Compute 96 upper-quartile radial profile values and divide the image by the profile selected for each pixel.
- (3) Compute τ from disk pixels only and threshold inside the disk.
- (4) Label four-connected foreground and measure every component area.
- (5) Discard components below 20 pixels; keep at most 64 by area.
- (6) Encode each retained label as compressed COCO RLE and write its unique `<image>_<rank>` identifier immediately.

Figure 2: Per-image inference procedure implemented by the final notebook.

Three invariants are checked at the output boundary. First, each row has exactly the two required fields. Second, identifiers are unique across the file. Third, decoding a sampled RLE recovers the source image dimensions and a nonempty mask. Connected components are disjoint by construction, so no pixel is assigned to two predicted instances from the same image. These checks catch packaging and serialization faults that a visually plausible overlay would not reveal.

3 EVALUATION AND MODEL SELECTION

Table 1 lists official public scores displayed for our Kaggle submissions. They are platform results, not synthetic local estimates. The 0.48 classical run remains the strongest verified route. Both later learned routes scored lower, so we froze them instead of selecting a more complex method on architectural appeal alone.

Table 1: Official public leaderboard results for our submissions.

Method	Reference	Score
Radial normalization + components	55113085	0.48
Torchvision Mask R-CNN	55354276	0.31
Public YOLO + U-Net refiner	55354612	0.34

The upstream public YOLO notebook reported a score of 0.69 for its original author. We do not attribute that score to our entry. Our adapted inference run is the 0.34 result shown above. This distinction is important because the final submission should be judged from reproducible entrant-owned outputs rather than a third party’s leaderboard record.

The classical method runs in approximately linear time in the number of image pixels, apart from component bookkeeping, and uses CPU memory for one image and one integer label map at

a time. Unit tests exercise synthetic filament recovery, disjoint-mask handling, connected-component scoring, and exact COCO RLE round trips. These tests establish implementation invariants; they are not substitutes for the official hidden evaluation.

3.1 Why the final route is classical

The three submitted routes answer different hypotheses. The radial method asks whether photometric correction plus explicit topology is sufficient. Mask R-CNN tests learned region proposals and instance masks. The public YOLO plus U-Net route tests a detector followed by learned refinement. Only the first route improved the entrant-owned public result to 0.48. We therefore use the public leaderboard solely as a selection signal among completed submissions and do not combine incompatible outputs after seeing their scores.

Table 2: Qualitative selection audit. Check marks denote properties of the submitted inference package, not additional leaderboard measurements.

Property	Radial	Mask R-CNN	YOLO+U-Net
No learned weights	✓	–	–
CPU-only final path	✓	–	–
No external download	✓	✓	✓
Best entrant score	✓	–	–

This choice also reduces submission risk. The selected notebook has no model checkpoint whose provenance or hash must be reconciled, no accelerator-specific kernel, and no training randomness. Its four runtime libraries are pinned in the root requirements file. Experimental dependencies remain in a separate file so that reproducing the final result does not silently install either learned stack.

4 ENGINEERING VALIDATION

4.1 Synthetic tests

Repository tests exercise the operations that are independent of unavailable ground truth. Radial normalization is tested on synthetic disks containing dark elongated regions. Component tests verify that disconnected regions remain separate and that small regions are filtered. Submission tests check that mask priority removes overlap, empty residual masks are dropped, and compressed RLE round-trips exactly. Metric tests verify that duplicate predictions reduce PQ, which guards against accidentally optimizing only matched-mask Dice.

Tests for the final deliverables parse the notebook as version-4 notebook JSON, require the complete inference and validation cells, and reject dependencies on the experimental computer-vision stack. They also assert exact version pins and check that the report preserves all three entrant submission references and the score-provenance warning. These are contract tests: they do not estimate hidden leaderboard performance, but they make drift between code, notebook, and report visible before submission.

4.2 Complexity and failure containment

Let $n = HW$ and let $B = 96$ be the fixed radial-bin count. Radius assignment, profile lookup, thresholding, connected-component labeling, and RLE traversal are linear in n . Computing quantiles requires additional selection work but does not change the practical image-wise memory bound. Component ranking depends on the number of labels, which cannot exceed n . The implementation deliberately processes and releases one image at a time, so dataset size increases runtime but not peak image storage.

A corrupt or unsupported file fails at its own read rather than contaminating a learned state. A scene with no retained component emits no instance row for that image; it does not manufacture a placeholder mask. A scene with excessive fragmentation is bounded by the 64-instance cap. These choices make failures observable and keep output size finite, although they can trade recall for robustness on unusually complex images.

4.3 Parameter roles and sensitivity

The final configuration exposes four conceptually distinct controls. The disk radius excludes off-disk background before any image statistic is calculated. The radial-bin count governs how locally the illumination profile follows center-to-limb variation: too few bins leave a residual gradient, while too many make the profile vulnerable to local structures and sparsely populated outer annuli. The darkness quantile controls foreground coverage, and its 0.95 ceiling prevents the adaptive threshold from admitting pixels that are not dark relative to their radial neighborhood. Minimum area and maximum instance count regularize topology after thresholding rather than changing pixel contrast.

These parameters were kept fixed for the final submission. We do not present a numerical ablation because no verified ablation table was recorded for the submitted classical run. Retrospectively fabricating one would confuse design rationale with measured evidence. A future controlled study should vary one parameter family at a time on a fixed image-level validation split and report both PQ and predicted-to-true instance-count ratio. The latter distinguishes a plausible foreground mask from excessive fragmentation or merging.

4.4 Submission-interface audit

The competition interface is treated as part of the method. File discovery is restricted to JPEG paths and sorted lexicographically, providing stable image order without assuming filesystem enumeration order. Each identifier combines the source stem with a one-based within-image rank. Masks are encoded through the COCO API in Fortran array order and the byte representation is converted to text before CSV serialization. The writer owns quoting and new-line handling, instead of concatenating fields manually.

The notebook validates the artifact after the writer closes it. It checks the header, ensures that data rows exist, rejects duplicate identifiers, rejects empty encoding strings, decodes a sample, and compares its shape to the source. This cannot prove that every instance is correct, but it rules out common zero-score failures: misspelled headers, duplicate keys, malformed RLE, transposed dimensions, and output written to the wrong location. Keeping these

checks next to inference makes the notebook closer to the exact submitted path.

5 DISCUSSION

Radial normalization is effective here as an inductive bias: solar observations have a known disk geometry and a large smooth photometric trend, while filaments are localized dark deviations. Encoding that structure explicitly reduces the amount of evidence the algorithm must infer from labeled examples. The method is also auditable because every accepted pixel can be traced to its radial bin, threshold, and connected component.

Simplicity is not semantic understanding. Connectivity cannot decide whether touching dark structures are distinct filaments, and it cannot bridge a faint interruption belonging to one physical object. Learned models remain promising when validated training data, reliable weights, and sufficient compute are available. For this submission, measured entrant-owned scores favor the transparent baseline, so the package reports that result rather than selecting the most sophisticated model.

6 LIMITATIONS AND REPRODUCIBILITY

Intensity connectivity is not a semantic model. A filament whose contrast is interrupted may fragment into several components, while neighboring filaments that touch after thresholding may merge. Bright active regions and acquisition artifacts can distort the radial profile. A fixed geometric disk can be less accurate than per-image limb estimation, and the minimum-area threshold can discard true fine structures. Finally, public leaderboard performance may not equal the final evaluation result.

The public release package comprises the MIT-licensed source, an exact pinned requirements file, and the final classical notebook in the kaggle directory. A reviewer attaches the official competition dataset and runs that notebook from the repository root; it writes `submission-classical.csv` to Kaggle’s standard working directory without a checkpoint or untracked input. Competition images and generated submissions are not redistributed in Git.

6.1 Reproduction protocol

The reviewer should begin in a fresh Kaggle session with the competition data attached and Internet access disabled after dependency availability is confirmed. Running the notebook from top to bottom performs five auditable phases: environment installation, source import, dataset discovery, inference, and output validation. The resulting CSV is written under the standard working directory. The displayed row count, identifier uniqueness assertion, decoded sample dimensions, and file preview should all complete before the artifact is submitted.

The repository intentionally excludes competition imagery, generated CSV files, and caches. Therefore a clean clone plus the organizer-provided dataset is the reproduction boundary. The classical result does not depend on a private split, checkpoint, secret, network service, or local absolute path. The final submitter must still verify that Kaggle exposes the same directory layout and that platform reference 55113085 corresponds to the 0.48 public score shown in this report.

6.2 Threats to validity

The main external-validity threat is leaderboard sampling. A public score is computed on the platform’s public partition and may reward choices that do not generalize to the private partition. We mitigate this only by selecting a simple method with physically motivated normalization and by reporting the public score as public; we do not claim a private or final score. A second threat is dataset shift across observatories, acquisition times, exposure settings, or disk sizes. The fixed 0.48 radius and contrast thresholds are properties of this submitted configuration rather than universal solar-image constants.

The principal construct-validity threat is that PQ combines recognition and segmentation quality, whereas visual inspection tends to emphasize only pixel coverage. A dark region can look reasonable while being split into too many instances, which increases false positives, or while merging distinct filaments, which increases false negatives. This is why our implementation tests duplicate penalties and disjointness, and why future validation should record instance count alongside overlap. Nonetheless, without organizer ground truth for the test data, this report cannot diagnose which PQ component dominates the 0.48 result.

Reproducibility has two remaining environmental threats. First, Kaggle may provide cached package wheels differently from a local Python environment; pinned versions specify the target but do not guarantee offline availability. Second, input directory names are platform-controlled. The notebook searches for the expected test JPEGs and prints the resolved path, but the final submitter must inspect that resolution rather than assume it. These are operational checks, not reasons to modify the algorithm after score selection.

6.3 Potential extensions

Several extensions follow directly from observed limitations but are deliberately left outside the submitted result. Estimating the limb per image could replace the fixed disk radius. Morphological linking guided by local orientation could bridge faint gaps without connecting all nearby dark pixels. A watershed or graph-cut stage could split touching candidates using intensity ridges. Finally, a learned instance model could use radial-normalized input as an additional channel, retaining the photometric prior while learning shape cues.

Any such extension should be evaluated on a frozen image-level split before a new platform submission. It should compare PQ, matched-mask overlap, false positive and false negative counts, and predicted-to-true instance ratio. Model weights and preprocessing would also need immutable hashes and a license audit. These proposals are future work only; none contributes to reference 55113085 or to the score claimed in this report.

AI assistance was used to help implement, test, and document repository materials. No AI-generated number is presented as an evaluation result. The entrant remains responsible for executing the notebook, verifying the three platform references, understanding the method, and submitting the final repository and report.

7 CONCLUSION

We present a complete, deterministic solar-filament instance-segmentation pipeline based on radial photometric normalization

and connected components. Its strongest verified public score is 0.48 under platform reference 55113085, above the two learned alternatives attempted by the entrant. The accompanying notebook, pinned minimal environment, artifact checks, tests, and explicit score provenance make the selected result straightforward to inspect and reproduce. Remaining scientific limitations are chiefly instance merging, fragmentation, and sensitivity to disk and contrast assumptions.

8 ACKNOWLEDGMENT

This challenge utilizes the MAGFiLO dataset for evaluation of segmentation algorithms. We thank the competition organizers, the National Solar Observatory, and the GONG program for providing the task and observations.

REFERENCES

- [1] Azim Ahmadzadeh, Dustin J. Kempton, Qin Li, and Alexei A. Pevtsov. 2026. Solar Filament Segmentation Challenge 2026. <https://www.kaggle.com/competitions/filament-segmentation-2026>.
- [2] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. 2019. Panoptic Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Long Beach, CA, USA, 9404–9413.
- [3] Tsung-Yi Lin, Michael Maire, Serge Belongie, et al. 2014. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision*. Springer, Cham, Switzerland, 740–755.